

ROS2

docker 환경 구축을 바로 하려면 오른쪽 클릭 🖱️

🗨️ ROS2는 왜 배우는가?

로봇 하나를 제어하려면 수많은 독립된 프로그램들이 동시에 돌아가야 한다.

ROS2는 이 프로그램들이 서로 데이터를 꼬임 없이 실시간으로 주고받을 수 있도록 통신 구조를 제공하는 로봇 전용 프레임워크다.

역할 (무엇을 하는가?)	실무 도구 및 예시 (무엇을 쓰는 가?)	핵심 요약
카메라/센서 데이터 수집	웹캠, RealSense, 라이다 (LiDAR)	주변 환경 데이터를 실시간으로 읽어옴
위치 추정	SLAM, Odometry	로봇이 자신의 현재 위치를 스스로 계산함
경로 계획	Navigation2 (Nav2)	목적지까지 장애물을 피해 가는 길을 찾음
로봇팔 제어	Movel2	다관절 로봇의 움직임과 각도를 정밀 계산함
모터 제어	ros2_control	하드웨어 모터에 직접 명령을 내리고 제어함
화면 시각화	RViz2	센서 데이터와 로봇의 상태를 화면에 3D로 보여줌
시뮬레이션	Gazebo Classic, MuJoCo	가상 공간에서 로봇을 안전하게 테스트함

ROS2 Humble은 2027년 5월까지 지원함. 이후부터는 Jazzy를 사용하게 될 수 있음.

Jazzy는 Ubuntu 24.04(Noble Numbat) 환경에서 동작하며, 2029년 5월까지 지원한다.

Node

Node는 ROS2에서 실행되는 하나의 프로그램 단위이다.

예를 들면 오른쪽 예시와 같다.

Node = 로봇 안에서 돌아가는 작은 프로그램 하나

```
/camera_node
/motor_controller_node
/lidar_node
/navigation_node
/robot_arm_controller_node
```

Topic

Topic은 Node끼리 연속적으로 데이터를 주고받는 통신 채널이다.

카메라 센서

라이다

속도 센서

/camera/image_raw

/scan

/cmd_vel

실무에서 가장 많이 쓰는 통신 방식 중 하나이다. (카메라, 라이다, 등)

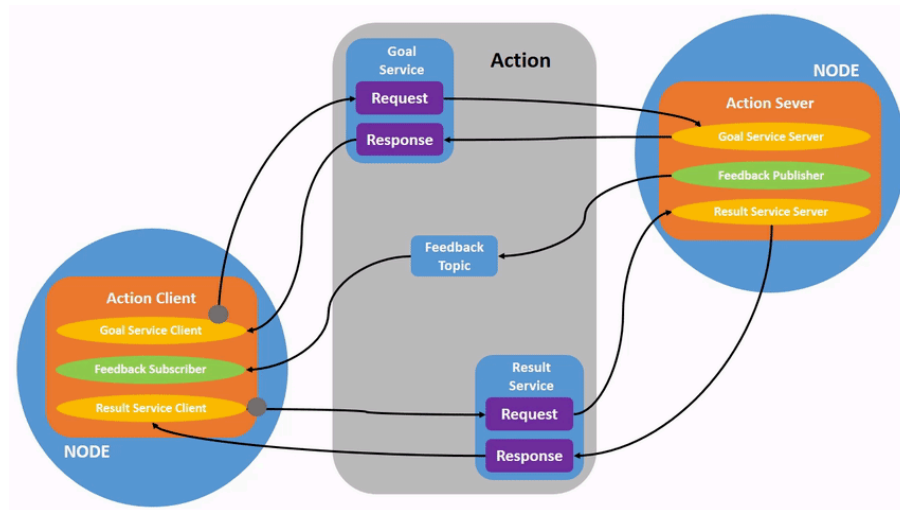
```
/camera_node    ---> /camera/image_raw    ---> /object_detection_node
# 카메라 노드      # 카메라 토픽      # 장애물 감지 노드
```

Topic / Service / Action

🗨️ 토픽, 서비스, 액션 이란?

- **토픽 (Topic):** 센서 데이터(카메라, 라이다)처럼 **쉬지 않고 일방적으로 들어오는 정보**를 읽을 때
- **서비스 (Service):** "센서 켜라", "좌표 리셋해라"처럼 **즉시 실행하고 끝나는 일회성 명령**을 내릴 때
- **액션 (Action):** "A 지점으로 이동해라"처럼 **중간 과정을 계속 체크(피드백)해야 할 때**

예시 영상



위 영상은 **액션**의 데이터 흐름에 대한 영상이다.

상단과 하단에 Goal, Result 라는 **서비스**와 중간에 Feedback **토픽**으로 이루어져있다.

서빙로봇을 예시를 들어 **Action**의 흐름을 이해해보자.

1) Goal Service (목표 지정)

- **상황** : 관제 시스템 → 로봇 (3번 테이블 앞으로 제육볶음 배달해라)
- **흐름** : **Client** (Request) → **Server** (Response: 접수 완료, 출발합니다!)
- **특징** : [일회성 서비스] - 명령 접수 여부 딱 한 번 확인

2) Feedback Topic (과정 보고)

- **상황** : 로봇 주행 중 실시간 상태 보고 (현재 50% 통과, 장애물 우회 중)
- **흐름** : **Server** (Publish) → **Client** (Subscribe: 관제 화면 표시)
- **특징** : [연속성 토픽] - 이동하는 내내 쉬지 않고 데이터 스트리밍

3) Result Service (최종 완료)

- **상황** : 로봇 목적지 도착 후 최종 임무 마감 및 결과 확인
- **흐름** : **Client** (Request) → **Server** (Response: 배달 성공 [SUCCESS])
- **특징** : [일회성 서비스] - 최종 성공/실패 여부 확인 후 **액션 완전 종료**

ROS2 dokcer 환경 구축

파일 구조

```

ros2_humble_cpp
├─ Dockerfile
├─ docker-compose.yml
└─ ros2_ws
    └─ ...

```

```

ros2_ws
├─ docker-compose.yml
└─ Dockerfile

```

Dockerfile

```

# =====
# ROS2 Humble C++ 개발용 Dockerfile
# 기준 이미지: osrf/ros:humble-desktop
# 목적:
#   - ROS2 Humble 기본 환경 제공
#   - C++ 빌드 도구 설치
#   - colcon 빌드 도구 설치
#   - ROS2 C++ 패키지 실습 준비
# =====

FROM osrf/ros:humble-desktop

# -----
# 1. 기본 셸 설정
# -----
SHELL ["/bin/bash", "-c"]

# -----
# 2. 환경 변수 설정
# -----
ENV DEBIAN_FRONTEND=noninteractive
ENV ROS_DISTRO=humble
ENV LANG=C.UTF-8
ENV LC_ALL=C.UTF-8

# -----
# 3. 개발 도구 설치
# -----
# build-essential : g++, make 등 C++ 빌드 도구
# cmake           : CMake 기반 빌드 도구
# git             : 소스 코드 다운로드
# nano/vim        : 컨테이너 내부 간단 편집용
# python3-colcon-common-extensions : ROS2 colcon 빌드 도구
# python3-rosdep  : ROS 패키지 의존성 관리 도구
# ros-humble-demo-nodes-cpp : C++ demo talker/listener 실습용
# ros-humble-demo-nodes-py : Python demo 비교용
RUN apt-get update && apt-get install -y \
    build-essential \
    cmake \
    git \
    nano \
    terminator \
    vim \
    tree \
    net-tools \

```

```

iputils-ping \
python3-pip \
python3-colcon-common-extensions \
python3-rosdep \
ros-humble-demo-nodes-cpp \
ros-humble-demo-nodes-py \
&& rm -rf /var/lib/apt/lists/*

# -----
# 4. rosdep 초기화
# -----
# rosdep init은 이미 되어 있거나 권한/네트워크 상태에 따라 실패할 수 있으므로
# 실패해도 이미지 빌드가 멈추지 않도록 처리합니다.
RUN rosdep init || true
RUN rosdep update || true

# -----
# 5. 작업 디렉터리 생성
# -----
WORKDIR /root/ros2_ws

# -----
# 6. bash 실행 시 ROS2 환경 자동 로드
# -----
RUN echo "source /opt/ros/humble/setup.bash" >> /root/.bashrc
RUN echo "if [ -f /root/ros2_ws/install/setup.bash ]; then source /root/ros2_ws/install/setup.bash; fi" >> /root/.bashrc

# -----
# 7. 기본 실행 명령
# -----
CMD ["/bin/bash"]

```

docker-compose.yml

```

services:
  ros2_humble_cpp:
    build:
      context: .
      dockerfile: Dockerfile

    container_name: ros2_humble_cpp

    # 컨테이너를 종료하지 않고 계속 실행하기 위한 설정
    stdin_open: true
    tty: true

    # ROS2 DDS 통신 테스트를 쉽게 하기 위해 host 네트워크 사용
    # Windows Docker Desktop에서는 Linux와 동작 방식이 다를 수 있지만,
    # 단일 컨테이너 학습 환경에서는 큰 문제가 없습니다.
    network_mode: host

    environment:
      - ROS_DOMAIN_ID=10

```

```
- RMW_IMPLEMENTATION=rmw_fastrtps_cpp
- DISPLAY=host.docker.internal:0.0
- QT_X11_NO_MITSHM=1
```

volumes:

```
# Windows 작업 폴더의 ros2_ws를 컨테이너 /root/ros2_ws에 연결
- ./ros2_ws:/root/ros2_ws
```

working_dir: /root/ros2_ws

command: /bin/bash

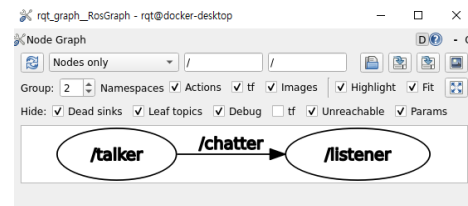
이후 도커 컨테이너 진입은 Docker 노션 참고. (build → up -d → exec) [👉 Docker 정리본](#)

1) 단순 통신 확인 코드

terminator로 창 나눠서 진행하면 됨. 주로 왼쪽 1개 / 오른쪽 2개로 쪼개서 사용함.

```
# ROS2 기본제공 노드
ros2 demo_nodes_cpp talker
ros2 demo_nodes_cpp listener
```

rqt_graph로 통신 방식 확인할 수 있음. 👉



2) 데이터 수동으로 넣기

```
# 터미널 1
ros2 topic pub /chatter std_msgs/msg/String "{data: 'Hello, ROS2'}"
# 터미널 2
ros2 demo_nodes_cpp listener
```

이렇게도 확인할 수 있음. (한 번만 실행)

```
ros2 topic pub --once /chatter std_msgs/msg/String "{data: 'Hello, ROS2'}"
```

3) 주요 코드들 모음

```
# 노드 확인
ros2 node list
ros2 node info /talker
ros2 node info /listener

# 토픽 확인
ros2 topic list
ros2 topic list -t
ros2 topic info /chatter
ros2 topic echo /chatter
```

```

ros2 topic hz /chatter
ros2 topic bw /chatter
ros2 topic pub --once /chatter std_msgs/msg/String "{data: 'Hello once'}"

# 메시지/서비스 구조 확인
ros2 interface show std_msgs/msg/String
ros2 interface show geometry_msgs/msg/Twist
ros2 interface show sensor_msgs/msg/JointState

# 서비스 확인
ros2 service list
ros2 service type /talker/get_parameters

# 파라미터 확인
ros2 param list /talker
ros2 param get /talker use_sim_time
ros2 param set /talker use_sim_time true

# 패키지 확인
ros2 pkg list
ros2 pkg list | grep demo
ros2 pkg executables demo_nodes_cpp

# 진단
ros2 doctor
ros2 doctor --report

```

★ 중요한 명령어 4개

```

ros2 node list           # 현재 실행 중인 모든 노드 목록 조회.
ros2 topic list -t       # 모든 토픽명 및 메시지 타입 동시 확인.
ros2 topic info /chatter # 지정 토픽의 데이터 구조와 연결 상태 점검.
ros2 topic echo /chatter # 지정 토픽 데이터의 실시간 흐름 모니터링.

```

📖 cpp_basic_nodes 패키지 만드는 방법

```

ros2 pkg create cpp_basic_nodes --build-type ament_cmake --dependencies rclcpp std_msgs

```

부분	의미
ros2 pkg create	ROS2 패키지 생성 명령
cpp_basic_nodes	패키지 이름
--build-type ament_cmake	C++용 CMake 기반 빌드 방식
--dependencies rclcpp std_msgs	사용할 ROS2 의존성

잘못 만들었다면, 만들어진 상위 폴더로 가서 아래 명령어 치면 됨.

```

rm -rf src/cpp_basic_nodes

```

📁 새로 생긴 파일 구조

```

└─ ros2_ws
    └─ src
        └─ cpp_basic_nodes
            └─ CMakeLists.txt
                └─ include
                    └─ cpp_basic_nodes
└─ package.xml
└─ src

```

CMakeLists.txt 파일 수정

`ament_package()` 코드 바로 위에 아래 내용을 붙여넣기 한다.

```

# -----
# simple_node 실행 파일 생성
# -----
add_executable(simple_node src/simple_node.cpp)

# simple_node가 사용할 ROS2 의존성 연결
ament_target_dependencies(simple_node
    rclcpp
    std_msgs
)

# ros2 run으로 실행할 수 있도록 install 경로 등록
install(TARGETS
    simple_node
    DESTINATION lib/${PROJECT_NAME}
)

```

코드 설명

설정 코드	한 줄 정의	무엇을 하는가?	왜 필요한가?
<code>add_executable</code>	프로그램 생성 명령이다.	C++ 소스 코드를 실행 파일로 빌드한다.	컴퓨터가 실행할 수 있는 실체(바이너리)를 만드는 과정이다.
<code>ament_target_dependencies</code>	라이브러리 연결 명령이다.	ROS2 핵심 기능(<code>rclcpp</code> , <code>std_msgs</code>)을 프로그램에 묶어준다.	이 설정이 없으면 코드의 <code>#include</code> 를 빌드 시스템이 인식하지 못한다.
<code>install(TARGETS ...)</code>	실행 경로 등록 명령이다.	완성된 프로그램을 ROS2 공식 폴더(<code>lib/</code>)로 이동시킨다.	이 처리를 해야 아무 터미널에서나 <code>ros2 run</code> 으로 실행할 수 있다.
<code>ament_package()</code>	최종 마감 도장이다.	"이 패키지의 모든 빌드 규칙 설정을 끝낸다"고 선언한다.	박스를 테이프로 닫는 마감선이므로, 위의 세 설정은 무조건 이보다 위에 와야 한다.

1) 패키지 컴파일 (빌드)

```
colcon build
```

해줘야 함.

- 이유: 내가 작성하거나 수정한 C++ 코드를 컴퓨터가 실행할 수 있는 프로그램으로 변환하기 위해 필수이다.

2) 빌드 환경 인식을 위한 소스 처리

이후, 아래 코드로 인식을 시켜줘야 함.

```
source install/setup.bash
```

이 처리를 하지 않으면 터미널이 새로 만든 패키지나 노드를 인식하지 못해 `Package not found` 에러가 발생한다.

3) 패키지 정상 등록 확인

이후, 패키지가 제대로 됐나 확인하려면 아래 입력

```
ros2 pkg list | grep cpp_basic_nodes
```